



10분 발표 전체 스토리라인 & 실전 스크립트

프로젝트: 전국 자동차 등록 현황 및 기업 FAQ 조회시스템 (수집-저장-조회 파이프라인)

[1장] 표지 및 인사

🕒 소요 시간: 30초

📄 슬라이드 내

용: 프로젝트 제목(전국 자동차 등록 현황 및 기업 FAQ 조회시스템), 팀원 이름

"안녕하세요. [팀 이름] 발표자 000입니다. 저희 팀은 공공 데이터와 기업 내부 데이터를 결합하여 통합적인 인사이트를 제공하는 '전국 자동차 등록 현황 및 기업 FAQ 조회시스템'을 개발했습니다. 수집부터 저장, GUI 구현까지의 과정을 지금부터 발표하겠습니다."

[2장] 프로젝트 개요 및 배경

🕒 소요 시간: 1분

📄 슬라이드 내

용: 01. 배경 (통계 통합 필요성), 02. 목표 (통합 시스템 구축), 03. 범위 (End-to-End), 04. 효과 (효율성 극대화)

"본 프로젝트는 두 가지 필요성에서 출발했습니다. 첫째는 국내 자동차 등록 대수가 2,600만 대를 넘어서며 발생하는 대용량 공공 데이터를 직관적으로 시각화하는 것이고, 둘째는 늘어나는 차량 관련 기업의 고객 문의 (FAQ) 데이터를 체계적으로 관리하는 것입니다. 저희는 이 두 데이터를 한곳에서 조회할 수 있는 '통합 파이프라인' 구축을 목표로 삼았습니다."

[3장] 시스템 아키텍처

🕒 소요 시간: 1분

📄 슬라이드 내

용: 수집(Python) → 저장(MySQL) → 표출(Streamlit) 파이프라인 흐름도

"전체 시스템 구조입니다. 데이터 엔지니어링의 정석적인 파이프라인을 따랐습니다. Python 기반의 *Beautiful Soup*과 *Selenium*으로 데이터를 수집했고, 이 데이터를 정제하여 MySQL 데이터베이스에 안정적으로 적재했습니다. 최종적으로는 사용자가 웹에서 실시간으로 데이터를 조회할 수 있도록 *Streamlit*을 활용해 GUI를 구현했습니다."

[4장] 데이터 수집 전략 1: Beautiful Soup

🕒 소요 시간: 1분

🖥️ 슬라이드 내

용: 국토교통부 통계누리 화면 캡처, BS4 파싱 개념도

"첫 번째 데이터 수집 단계입니다. 전국 자동차 등록 현황은 국토교통부 통계누리의 데이터를 활용했습니다. 이 페이지는 구조가 고정되어 있는 정적 페이지였기 때문에, HTTP 요청 속도가 빠르고 가벼운 *Beautiful Soup 4*를 사용했습니다. 이를 통해 과거 수개년 동안의 지역별, 차종별 등록 대수 *Raw* 데이터를 빠른 속도로 일괄 스크래핑했습니다."

[5장] 데이터 수집 전략 2: Selenium

🕒 소요 시간: 1분

🖥️ 슬라이드 내

용: 기업 FAQ 페이지 '더보기' 버튼 클릭하는 봇 이미지 또는 캡처

"두 번째는 기업 FAQ 데이터 수집입니다. 대다수의 기업 FAQ 페이지는 사용자가 '더보기' 버튼을 누르거나 자바스크립트가 실행되어야 데이터가 나오는 동적 구조를 가지고 있었습니다. 정적 크롤러로는 한계가 있어, 웹 브라우저를 직접 제어하는 *Selenium*을 도입했습니다. *Headless* 모드를 적용해 백그라운드에서 자동으로 버튼 클릭과 스크롤을 수행하며 데이터를 누락 없이 확보했습니다."

[6장] 데이터베이스(MySQL) 설계

🕒 소요 시간: 1분 30초

🖥️ 슬라이드 내

용: ERD (테이블 간의 관계도 - 자동차 통계 테이블, FAQ 테이블, 마스터 테이블)

"수집한 데이터를 저장하기 위한 *MySQL DB* 설계 단계입니다. 데이터의 중복을 방지하고 무결성을 유지하기 위해 3차 정규화를 진행했습니다. 자동차 등록 현황 테이블과 FAQ 테이블을 독립적으로 구성하고, '지역 코드'와 '차종 코드' 같은 반복되는 데이터는 공통 마스터 테이블로 분리하여 관계(*FK*)를 설정했습니다. 또한, *Pandas*를 활용해 결측치와 특수문자를 정제한 후 데이터 타입에 맞게 안전하게 *Insert* 했습니다."

[7장] 백엔드 연동 로직

🕒 소요 시간: 1분

🖥️ 슬라이드 내

용: PyMySQL, SQLAlchemy 엔진을 활용한 파이썬-DB 데이터 흐름 아키텍처

"이렇게 저장된 DB를 사용자 화면과 연결하는 로직입니다. 파이썬의 *PyMySQL*과 *SQLAlchemy* 라이브러리를 활용해 데이터베이스 커넥션 엔진을 구축했습니다. 사용자가 화면에서 특정 필터를 선택하면, 프로그램 내부에서 동적으로 SQL 쿼리문이 생성되어 MySQL로 전달되고, 필요한 데이터만 정확하게 *Fetch* 해오는 방식으로 시스템 연동을 완료했습니다."

[8장] 기능 시연 1: 자동차 현황 대시보드

🕒 소요 시간: 1분 30초

🖥️ 슬라이드 내

용: Streamlit 실제 구동 화면 (대시보드 필터링 및 차트 시각화 캡처)

"가장 중요한 GUI 구현 결과물입니다. 파이썬 기반의 웹 프레임워크인 *Streamlit*을 사용했습니다. 첫 번째 탭인 '자동차 등록 현황 대시보드'에서는 사용자가 연도, 지역, 차종을 선택하면 그에 맞는 통계 데이터가 실시간으로 반영됩니다. 보시는 화면처럼 경기도가 가장 높은 등록 수를 기록하고 있는 등, 복잡한 통계 수치를 바(Bar) 차트와 라인(Line) 차트로 직관적으로 시각화했습니다."

[9장] 기능 시연 2: 기업 FAQ 조회 시스템

🕒 소요 시간: 1분

🖥️ 슬라이드 내

용: Streamlit FAQ 검색창 및 카테고리 필터링 구동 화면

"두 번째 탭인 '기업 FAQ 조회 시스템'입니다. 수집된 방대한 FAQ 데이터를 카테고리별(예: AS, 차량구매, 계약 등)로 필터링할 수 있으며, 상단의 검색창에 키워드를 입력하면 해당 단어가 포함된 질문과 답변만 테이블 형태로 즉시 필터링되어 출력됩니다. 이를 통해 사용자는 원하는 답변을 몇 초 만에 찾아낼 수 있습니다."

[10장] 트러블 슈팅

 소요 시간: 1분

슬라이드 내

용: [문제점] 동적 로딩으로 인한 DB 부하 → [해결책] @st.cache_data 데코레이터 적용

"개발 과정에서의 핵심 트러블 슈팅입니다. *Streamlit*은 사용자가 버튼을 누르거나 필터를 바꿀 때마다 전체 소스코드가 처음부터 재실행되는 특징이 있습니다. 이 때문에 매번 DB에 다시 접속하여 쿼리를 날리느라 심각한 로딩 지연이 발생했습니다. 저희 팀은 이를 해결하기 위해 캐싱 데코레이터인 @st.cache_data를 도입했습니다. 한 번 조회한 데이터는 메모리에 저장해 두고 재사용함으로써 UI 반응 속도를 기존 대비 80% 이상 개선했습니다."

[11장] 결론 및 향후 과제

 소요 시간: 30초

슬라이드 내

용: 향후 발전 과제 파트 (Airflow 수집 자동화, LLM 기반 FAQ 챗봇 도입)

"마지막으로 결론입니다. 이번 프로젝트를 통해 데이터 수집부터 시각화까지의 엔드투엔드 파이프라인을 성공적으로 구축했습니다. 현재는 수동으로 크롤러를 돌리지만, 향후 *Airflow*를 도입해 수집을 자동화하고, FAQ 데이터를 기반으로 한 AI 챗봇 기능까지 도입하는 것이 최종 목표입니다."

[12장] Q&A 및 마무리

 소요 시간: 마무리

슬라이드 내

용: "감사합니다. Q&A" 및 팀 프로젝트 Repository 링크

"이상으로 발표를 마치겠습니다. 질문 있으시면 답변해 드리겠습니다. 감사합니다."